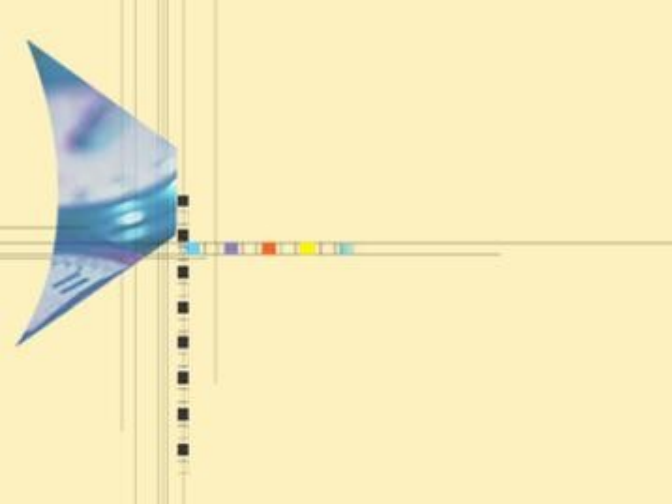




第 7 讲

NP 完全问题

——Computability Theory



本章内容

- 7.1 引言
 - 7.1.1 问题复杂性类
 - 7.1.2 算法复杂度与问题复杂度关系
 - 7.1.3 可计算问题、判定问题
 - 7.1.4 确定图灵机模型、确定算法
 - 7.1.5 不确定图灵机模型、不确定算法
- 7.2 P 类、NP 类
 - 7.2.1 P 类定义、性质及举例
 - 7.2.2 NP 类定义及举例
 - 7.2.3 P 类和 NP 类的关系、COOK 定理



本章内容

- 7.3. 多项式归约及 NP- 完全类
 - 7.3.1 多项式归约定义
 - 7.3.2 多项式归约性质
 - 7.3.3 NP- 难类、 NP- 完全类
 - 7.3.4 NP- 完全性证明
 - 7.3.5 其他问题复杂性类



7.1 引言

- 7.1.1 问题复杂性类
- 7.1.2 算法复杂度与问题复杂度关系
- 7.1.3 可计算问题、判定问题
- 7.1.4 确定图灵机模型、确定算法
- 7.1.5 不确定图灵机模型、不确定算法



7.1.1 问题复杂性类

- 设 Π 是任意问题，如果对问题 Π 存在一个算法，它的时间复杂性是 $O(n^k)$ ，其中 n 是输入大小， k 是非负整数，我们说存在着求解问题 n 的**多项式时间算法**。这类算法在**可以接受的时间内**实现问题求解， e.g., 排序、串匹配、矩阵相乘。
- 但是，现实世界中的许多有趣问题并不属于这个范畴，因为求解这些问题所需要的时间量要用**指数和超指数函数**（如 2^n 和 $n!$ ）来测度。随着问题规模的增长而**快速增长**。
- 在计算机科学界已达成这样的共识，认为存在多项式时间算法的问题是**易求解**的，而对于那些不大可能存在多项式时间算法的问题是**难解**的。

易解问题与难解问题的主要区别

在学术界已达成这样的共识：把多项式时间复杂性作为易解问题与难解问题的分界线，主要原因有：

- 1) 多项式函数与指数函数的增长率有本质差别

问题规模 n	多项式函数					指数函数	
	$\log n$	n	$n \log n$	n^2	n^3	2^n	$n!$
1	0	1	0.0	1	1	2	1
10	3.3	10	33.2	100	1000	1024	3628800
20	4.3	20	86.4	400	8000	1048376	2.4E18
50	5.6	50	282.2	2500	125000	1.0E15	3.0E64
100	6.6	100	664.4	10000	1000000	1.3E30	9.3E157

2) 计算机性能的提高对易解问题与难解问题算法的影响

假设求解同一个问题有 5 个算法 A1~A5，时间复杂度 $T(n)$ 如下表，假定计算机 C2 的速度是计算机 C1 的 10 倍。下表给出了在相同时间内不同算法能处理的问题规模情况：

$T(n)$	n	n'	变化	n'/n
$10n$	1000	10000	$n'=10n$	10
$20n$	500	5000	$n'=10n$	10
$5n\log n$	250	1842	$\sqrt{10} \ n < n' < 10n$	7.37
$2n^2$	70	223	$\sqrt{10} \ n$	3.16
2^n	13	16	$n'=n+\log 10 \approx n+3$	≈ 1

3) 多项式时间复杂性忽略了系数，不影响易解问题与难解问题的划分

问题规模 n	多项式函数			指数函数	
	n^8	$10^8 n$	n^{1000}	1.1^n	$2^{0.01n}$
5	390625	5×10^8	5^{1000}	1.611	1.035
10	10^8	10^9	10^{1000}	2.594	1.072
100	10^{16}	10^{10}	10^{2000}	13780.6	2
1000	10^{24}	10^{11}	10^{3000}	2.47×10^{41}	1024

观察结论： $n \leq 100$ 时，（不自然的）多项式函数值大于指数函数值，但 n 充分大时，指数函数仍然超过多项式函数。

难解问题

- 这一类含有许许多多问题，其中还包含了数百个著名的问题，它们有一个共同的特性，即如果它们中的一个是多项式可解的，那么所有其他的问题也是多项式可解的。
- 此外，现存的求解这些问题的算法的运行时间，对于中等大小的输入也要用几百或几千年来测度。

7.1.2 算法复杂性与问题复杂性关系

- 算法作为求解问题的方法，可以求解现实世界中很多问题，但有些问题仍然无法用任何算法求解，有些问题虽然有算法可以求解，但需要太长的运行时间或太大的存储空间
- 算法的复杂性是指解决问题的一个具体的算法的执行时间，这是算法的性质；问题的复杂性是指这个问题本身的复杂程度。
- 很明显，问题复杂度是固有的，算法复杂度是依赖于很多现有条件的（包括逻辑条件和物理条件，前者是设计方案能力，后者是实现能力）

7.1.3 可计算问题、判定问题

- 计算复杂性理论有两个基本论题：Church-Turing 图灵论题和 Cook-Karp 库克论题
- Church-Turing 论题：一个问题时可计算的当且仅当它在图灵机上经过有限次计算得到正确的结果。这个论题把人类所面临的问题分为两类：一类是可计算的，另一类是不可计算的。但“有限次计算”是一个宽松的条件
- Cook-Karp 论题：一个问题实际可计算的当且仅当它在图灵机上经过多项式时间（步数）计算得到正确的结果。Cook-Karp 论题将可计算问题类进一步划分成两类：一类是实际可计算的，另一类是实际不可计算的

7.1.3 可计算问题、判定问题

- **难解问题**：虽没找到多项式时间算法，但按指数时间算法的难解问题至少在理论上可以用计算机求解。
- 但有些问题不论耗多少时间也不能用计算机求解。
- 不能用计算机求解（不论耗费多少时间）的问题称为**不可解问题**。

判定问题和最优化问题

- 判定问题：
 - 在研究 NP 完全性理论时，我们很容易重述一个问题使它的解只有两个结论：yes 或 no，在这种情况下，称问题为判定问题。
- 最优化问题：
 - 与此相对照，最优化问题是关心某个量的最大化或最小化的问题。在前面的章节中，已经遇到过大量的最优化问题，像找出一张表中的最大或最小元素的问题，在有向图中寻找最短路径问题和计算一个无向图的最小生成树的问题。
- 如果我们有一个求解判定问题的有效算法，那么很容易把它变成求解与它相对应的最优化问题的算法。反之亦然。

例子 ELEMENT UNIQUENESS 问题



设 S 是一个实数序列，ELEMENT UNIQUENESS 问题为，是否 S 中的所有数都是不同的。

- 判定问题：ELEMENT UNIQUENESS.

输入：一个整数序列 S

问题：在 S 中存在两个相等的元素吗？

- 最优问题：ELEMENT COUNT

输入：一个整数序列 S

输出：一个在 S 中频度最高的元素。

这个问题可以用显而易见的方法在最优的时间 $O(n \log n)$ 解决，这意味着它是易解的。

例子 COLORING 问题 (图着色问题)



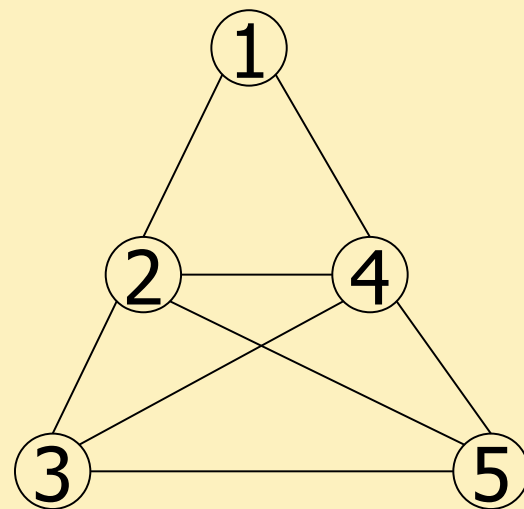
给出一个无向图 $G = (V, E)$, 用 k 种颜色对 G 着色是这样的问题: 对于 V 中的每一个顶点用 k 种颜色中的一种对它着色, 使图中没有两个邻接顶点有相同的颜色。

这个问题是难解的。



例子 COLORING 问题 (图着色问题)

- 输入：(着色数) k , (节点数) 5 , (图的边) $(1,2)(1,4)...$
- 写出长度为 n (节点数) 的字符串, 例如, RGRBG, RGRB, RBYGO, RGRBY, R%*\$@ 等, 至少有 k^n 个不同着色情况。
- 找到符合要求 (任何两个邻接顶点颜色不同) 的最小的 k 值



例子 2 (Continue)



判定问题 : COLORING

输入：一个无向图 $G = (V, E)$ 和一个正整数 $k \geq 1$ 。

问题： G 可以 k 着色吗？即 G 最少可以用 k 种颜色着色吗？

- 这个问题有一个最优形式是，对一个图着色，使图中没有两个邻接的顶点有相同的颜色，所需要的最少颜色数是多少？这个数记为 $\chi(G)$ ，称为 G 的色数。

- 最优化问题 : CHROMATIC NUMBER

输入：一个无向图 $G = (V, E)$ 。

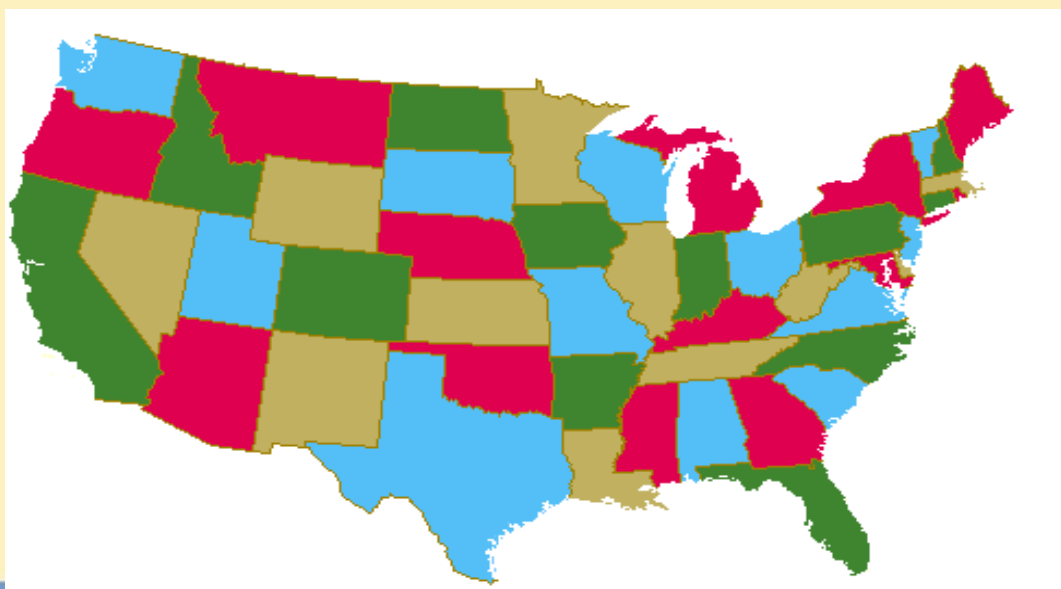
输出： G 的色数。



四色猜想

- 地图四色定理 (Four color theorem) 最先是由一位叫古德里 (Francis Guthrie) 的英国大学生提出来的。

四色问题的内容是：“任何一张地图只用四种颜色就能使具有共同边界的国家着上不同的颜色。”



四色猜想

- 1852 年，刚从伦敦大学毕业的 Francis Guthrie 提出了四色猜想。
- 1878 年著名的英国数学家 Cayley 向数学界征求解答。
- 此后数学家 Heawood 花费了毕生的精力致力于四色研究，于 1890 年证明了五色定理（每个平面图都是 5 顶点可着色的）。
- 直到 1976 年 6 月，美国数学家 K. Appel 与 W. Haken，在 3 台不同的电子计算机上，用了 1200 小时，作了 100 亿判断，才终于完成了“四色猜想”的计算机证明。

例子 3



给出一个无向图 $G=(V,E)$ ，对于某个正整数 k ， G 中大小为 k 的团集，是指 G 中有 k 个顶点的一个完全子图。团集问题是问一个无向图是否包含一个预定大小的团集。

- 判定问题：CLIQUE.

输入：一个无向图 $G=(V,E)$ 和一个正整数 k 。

问题： G 有大小为 k 的团集吗？

- 最优化问题：MAX-CLIQUE.

输入：一个无向图 $G=(V,E)$ 。

输出：一个正整数 k ，它是 G 中最大团集的大小



- 如果我们有一个求解判定问题的有效算法，那么很容易把它变成求解与它相对应的最优化问题的算法。
- 例如，我们有一个求解图着色判定问题的算法 A，则可以用二分搜索并且把算法 A 作为子程序来找出图 G 的色数。很清楚， $1 \leq x(G) \leq n$ ，这里 n 是 G 中顶点数，因此仅用 $O(\log n)$ 次调用算法 A 就可以找到 G 的色数。由于我们正处理多项式时间的算法， $\log n$ 因子是不重要的。
- 因为这个理由，在 NP 完全问题的研究中，甚至在一般意义上的计算复杂性或可计算性的研究中，把注意力限制在判定问题上会比较容易一些

7.1.4 确定图灵机模型、确定算法

- 图灵机是一种数学计算模型，它定义了一个抽象机器，该抽象机器根据规则表来操纵带子上的符号。尽管该模型很简单，但是在任何给定计算机算法的情况下，都可以构建出模拟该算法逻辑的图灵机。
- 简单点说，图灵机就是一个模拟算法运行的抽象机器。它是这样定义的：
 - 有一个无限长度的磁带，这个磁带被分成了一个接一个的单元格，磁带被用于写入字母和符号。
 - 一个读写磁带的磁头，这个磁头负责控制堆磁带的写入和左右移动。
 - 一个状态寄存器，用来存储图灵机的状态。
 - 一个指令表，可以根据机器当前所处的状态和磁带上当前的符号，指示机器进行特定的操作。比如：擦除或者写入一个符号、向左或者向右移动磁头。
 - 可以看到整个图灵机基本上模拟了程序的执行步骤。



确定图灵机模型

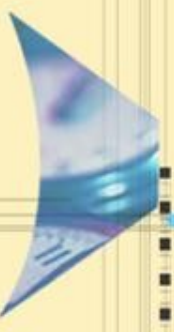
- 在确定性图灵机（DTM）中，其控制规则规定了在任何给定情况下最多只能执行一个动作。
- 确定性图灵机具有转换功能，对于磁带头下的给定状态和符号，该转换功能指定了三件事：
- 要写入磁带的符号，头部应移动的方向（向左，向右或都不向），以及有限控制的后续状态。
- 例如，状态 3 的磁带上的 X 可能会使 DTM 在磁带上写 Y，将磁头向右移动一个位置，然后切换到状态 5。

确定算法

- 定义 确定性算法
 - 设 A 是求解问题 Π 的一个算法，如果在展示问题 Π 的一个实例时，在整个执行过程中，每一步都只有一种选择，则称 A 是确定性算法。因此如果对于同样的输入，实例一遍又一遍地执行，它的输出从不改变。
 - 特点：对同一输入实例，运行算法 A ，所得结果是一样的。

7.1.5 不确定图灵机模型、不确定算法

- 不确定图灵机模型：
 - 在理论计算机科学中，不确定性图灵机（NTM）是一种理论计算模型，其控制规则在某些给定情况下指定了多个可能的动作。也就是说，NTM 的下一个状态不是完全由其动作和它所看到的当前符号决定的（不同于确定性图灵机）。
 - 例如，状态 3 的磁带上的 X 可能允许 NTM：
 - 输入 Y，向右移动，然后切换到状态 5 或者写一个 X，向左移动，并停留在状态 3。



不确定性算法

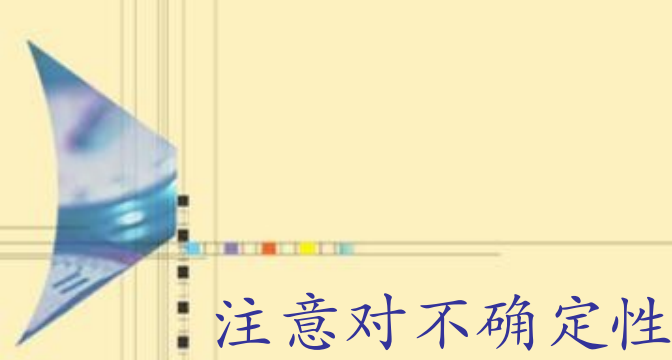
- 对于输入 x , 一个不确定性算法由下列两个阶段组成：
 - 猜测阶段
 - 验证阶段

不确定性算法——猜测阶段

- 在这个阶段产生一个任意字符串 y ，它可能对应于输入实例的一个解，也可以不对应解。
- 事实上，它甚至可能不是所求解的合适形式，它可能在不确定性算法的不同次运行中不同。它仅仅要求在多项式步数内产生这个串，即在 $O(n^i)$ 时间内，这里 $n=|x|$ ， i 是非负整数。对于许多问题，这一阶段可以在线性时间内完成。

不确定性算法——验证阶段

- 在这个阶段，一个不确定性算法验证两件事
 - 首先，它检查产生的解串 y 是否有合适的形式，
 - 如果不是，则算法停下并回答 no;
 - 另一方面，如果 y 是合适形式，那么算法继续检查它是否是问题实例 x 的解，如果它确实是实例 x 的解，那么它停下并且回答 yes，否则它停下并回答 no。
- 我们也要求这个阶段在多项式步数内完成，即在 $O(n^j)$ 时间内，这里 j 是一个非负整数。



注意对不确定性算法输出 yes/no 的理解：

- 若输出 no，并不意味着不存在一个满足要求的解，因为猜测可能不正确；若输出 yes，则意味着对于该判定问题的某一输入实例，至少存在一个满足要求的解。



不确定的算法——伪代码

- **Void** nondetA(String input)
 String s=genCertif();
 boolean checkOK=verifyA(input,s)
 if (checkOK)
 Output “yes”
 return
- **checOK** 为 **false** 时不作反应。



不确定性算法

- 定义 (**不确定性算法**): 设 A 是求解问题 Π 的一个算法, 如果算法 A 以如下 **猜测 + 验证** 的方式工作, 称算法 A 为不确定性 (nondeterminism) 算法:
- **猜测阶段**: 对问题的输入实例产生一个任意字符串 y , 在算法的每次运行, y 可能不同, 因此猜测是以不确定的形式工作。这个工作一般可以在 **线性时间内** 完成。
- **验证阶段**: 在这个阶段, 用一个确定性算法验证两件事: 首先验证猜测的 y 是否是合适的形式, 若不是, 则算法停下并回答 no; 若是合适形式, 则继续检查它是否是问题 x 的解, 如果确实是 x 的解, 则停下并回答 yes, 否则停下并回答 no。要求验证阶段在 **多项式时间内** 完成。



7.2 P 类、NP 类

- 7.2.1 P 类定义、性质及举例
- 7.2.2 NP 类定义及举例
- 7.2.3 P 类和 NP 类的关系、COOK 定理



7.2.1 P 类

定义 10.2 P 类问题

- 判定问题的 P 类由这样的判定问题组成，它们的 yes/no 解可以用确定性算法在运行多项式步数内，例如在 $O(n^k)$ 步内得到，其中 k 是某个非负整数， n 是输入大小。
- 也就是说：如果对于某个判定问题 Π ，存在一个非负整数 k ，对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行一个确定性算法，得到 yes 或 no 的答案，则称该判定问题 Π 是一个 P(Polynomial) 类问题。
(Polynomial 多项式)
- 事实上，所有易解问题都是 P 类问题。

P 类问题例子

排序问题:

给出一个 n 个整数的表，它们是否按非降序排列？

不相交集问题:

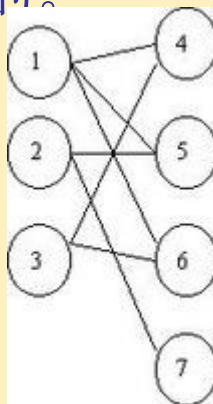
给出两个整数集合，它们的交集是否为空？

最短路径问题:

给出一个边上有正权的有向图 $G = (V, E)$ ，两个特异的顶点 $s, t \in V$ 和一个正整数 k ，在 s 到 t 间是否存在一条路径，它的长度最多是 k 。

P 类问题例子

- 2 着色问题：
 - 给出一个无向图 G ，它是否是 2 可着色的？即它的顶点是否可仅用两种颜色着色，使两个邻接顶点不会分配相同的颜色？注意，当且仅当 G 是二分图，即当且仅当它不包含奇数长的回路时，它是 2 可着色的。



- 2 可满足问题：
 - 给出一个合取范式 (CNF) 形式的布尔表达式 f ，这里每个子句恰好由两个文字组成，问 f 是可满足的吗？

P 类问题性质

- 如果对于任意问题 C ， Π 的补也在 C 中，我们说问题类 $\Pi \in C$ 在补运算下是封闭的。
 - 例如，2 着色问题的补可以陈述如下：给出一个图 G ，它是不 2 可着色的吗？我们称这个问题为 NOT-2-COLOR 问题。它是属于 P 的。



定理

- P 类问题在补运算下是封闭的。



7.2.2 NP 类

- **NP 类**由这样的问题 Π 组成，对于这些问题存在一个确定性算法 A ，该算法在对 Π 的一个实例展示一个断言解时，它能在**多项式时间内验证解的正确性**。即如果断言解导致答案是 yes，就存在一种方法可以在多项式时间内**验证这个解**。

NP 类

- 至于一个(不确定性)算法的**运行时间**,它仅仅是两个运行时间的和:一个是猜测阶段的时间,另一个是验证阶段的时间。因此它是 $O(n^i)+O(n^j)=O(n^k)$, k 是某个非负整数。



定义

- 判定问题类 NP 由这样的判定问题组成:对于它们存在着多项式时间内运行的不确定性算法。



NP 类问题

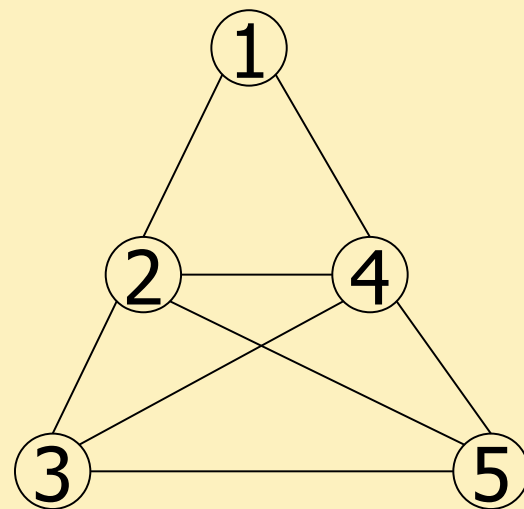
- 定义 (**NP 类问题**): 如果对于判定问题 Π , 存在一个非负整数 k , 对于输入规模为 n 的实例, 能够以 $O(n^k)$ 的时间运行一个**不确定性算法**, 得到 **yes/no** 的答案, 则该判断问题 Π 是一个 NP(**n**ondeterministic **p**olynomial) 类问题。

※注意: NP 类问题是对于判定问题定义的, 事实上, 可以在多项式时间内应用非确定性算法解决的所有问题都属于 NP 类问题。



例子 (图着色问题)

- Input: (着色数) k , (节点数) 5 , (图的边) $(1,2)(1,4)...$
- Guessing 指写出长度为 n (节点数) 的字符串, 例如, RGRBG, RGRB, RBYGO, RGRBY, R%*\$@ 等. 至少有 k^n 个“guessings”。
- Checking 指检查一 guessed 字符串是否合法及是否是一个 k - 着色。
- 如果写字符串的时间是 $O(p_1(n))$, checking 时间是 $O(p_2(n))$; 而且 $p_1(n), p_2(n)$ 是 n 的多项式 (从而也是输入长度的多项式), 则该不确定算法有多项式时间。
- 如果输入的图能 k 着色则不确定算法停机并给出 yes 回答。



例子



考虑问题 COLORING，我们用两种方法证明这个问题属于 NP 类。

- 方法 1:
 - 设 I 是 COLORING 问题的一个实例， s 被宣称为 I 的解。
 - 容易建立一个确定性算法来验证，是否确实是 I 的解。
 - 从 NP 类的非形式定义可得 COLORING 问题属于 NP 类。



例子

方法 2:

建立不确定性算法

- 当图 G 用编码表示后，一个算法 A 可以很容易地构建并运作如下
- 首先通过对顶点集合产生一个任意的颜色指派以“猜测”一个解 First。
- 接着， A 验证这个指派是否是有效的指派，如果它是一个有效的指派，那么 A 停下并且回答 yes，否则它停下并回答 no。
 - 请注意，根据不确定性算法的定义，仅当对问题的实例回答是 yes 时， A 回答 yes。
 - 其次是关于需要的运行时间， A 在猜测和验证两个阶段总共花费不多于多项式时间。

7.2.3 P 类和 NP 类的关系、 COOK 定理

- 1) 若问题 Π 属于 P 类，则存在一个多项式时间的确定性算法，对它进行判定或求解；显然，也可以构造一个多项式时间的非确定性算法，来验证解的正确性，因此，问题也属 NP 类。因此，显然有

$$P \subseteq NP$$

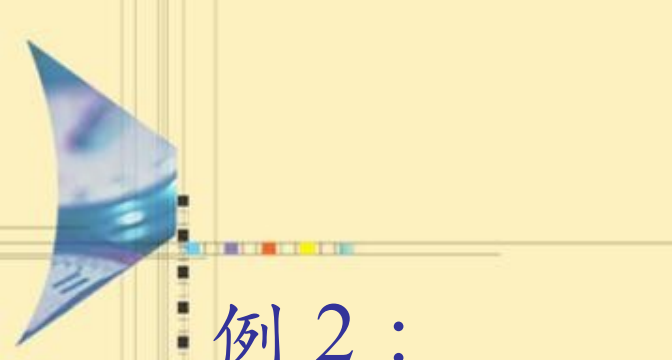
- 2) 若问题 Π 属于 NP 类，则存在一个多项式时间的非确定性算法，来猜测并验证它的解；但不一定能构造一个多项式时间的确定性算法，来对它进行求解或判定。

- 因此，人们猜测 $P \neq NP$ ，但是否成立，至今未得到证明。
- $P=NP?$ 是计算机科学中最大的问题之一

世界七大数学难题 ----NP 完全问题

例 1 :

- 在一个周六的晚上，你参加了一个盛大的晚会。由于感到局促不安，你想知道这一大厅中是否有你已经认识的人。宴会的主人向你提议说，你一定认识那位正在甜点盘附近角落的女士罗丝。不费一秒钟，你就能向那里扫视，并且发现宴会的主人是正确的。然而，如果没有这样的暗示，你就必须环顾整个大厅，一个个地审视每一个人，看是否有你认识的人。



例 2 :

- 如果某人告诉你，数 13717421 可以写成两个较小的数的乘积，你可能不知道是否应该相信他，但是如果他告诉你它可以分解为 3607 乘上 3803，那么你就可以用一个袖珍计算器容易验证这是对的。
- 生成问题的一个解通常比验证一个给定的解时间花费要多得多。

NP=P ? 的猜想

- 人们发现，所有的完全多项式非确定性问题，都可以转换为一类叫做满足性问题的逻辑运算问题。既然这类问题的所有可能答案，都可以在多项式时间内计算，人们于是就猜想，是否这类问题，存在一个确定性算法，可以在多项式时间内，直接算出或是搜寻出正确的答案呢？这就是著名的 NP=P ? 的猜想。
- 不管我们编写程序是否灵巧，判定一个答案是可以很快利用内部知识来验证，还是没有这样的提示而需要花费大量时间来求解，被看作逻辑和计算机科学中最突出的问题之一。它是斯蒂文·考克于 1971 年陈述的。

可满足性问题

- 可满足性问题即合取范式的可满足性问题，来源于许多实际的逻辑推理的应用。合取范式形如 $A_1 \wedge A_2 \wedge \dots \wedge A_n$ ，其中子句 $A_i (1 \leq i \leq n)$ 形如： $a_1 \vee a_2 \vee \dots \vee a_k$ ，其中 a_j 称为文字，为布尔变量或它的否定。一个子句是文字的析取。
- SAT 问题是指：是否存在一组对所有布尔变量的赋值，使得整个合取范式为真？例如
$$f = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3 \vee x_4 \vee \bar{x}_5) \wedge (x_1 \vee \bar{x}_3 \vee x_4)$$

当 x_1 和 x_3 都为真、其余文字任意赋值时， f 值为真。

可满足性问题

- 判定问题：SATISFIABILITY.

输入：一个合取范式的布尔公式 f 。

问题： f 是可满足的吗？



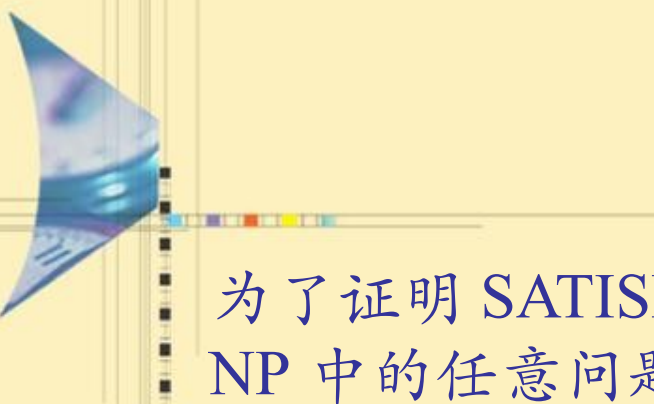
定理 10.2

- SATISFIABILITY 问题是 NP 完全的
- 实际上，可满足性问题被证明是 NP 完全的第一个问题。



可满足性问题

- **可满足性**（英语：Satisfiability）问题又叫布林可满足性问题（**Boolean satisfiability problem**；简写**SAT**）属于判定问题，它是历史上第一个被证明 **NP 完全问题**。
- 1971 年，美国的 Cook 证明了 **Cook 定理**：布尔表达式的可满足性 (SAT) 问题是 **NP 完全**的。



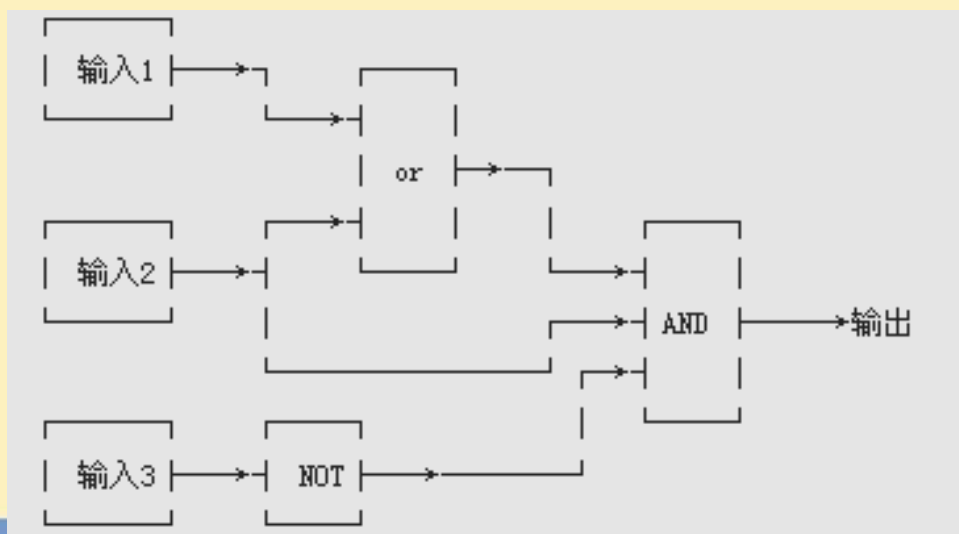
为了证明 SATISFIABILITY 是 NP 完全的，必须证明对 NP 中的任意问题 $\square$ ，有 $\square \leq_{\text{poly}} \text{SATISFIABILITY}$ 。假设 $\square$ 的实例 I 的规模是 n ，在 $p(n)$ 的判定时间里最多有 $cp(n)$ 个动作，因此可以用 $cp(n)$ 时间构造布尔表达式 f ，也就是说 $\square \leq_{\text{poly}} \text{SATISFIABILITY}$

这就是著名的 **Cook 定理**



直观描述

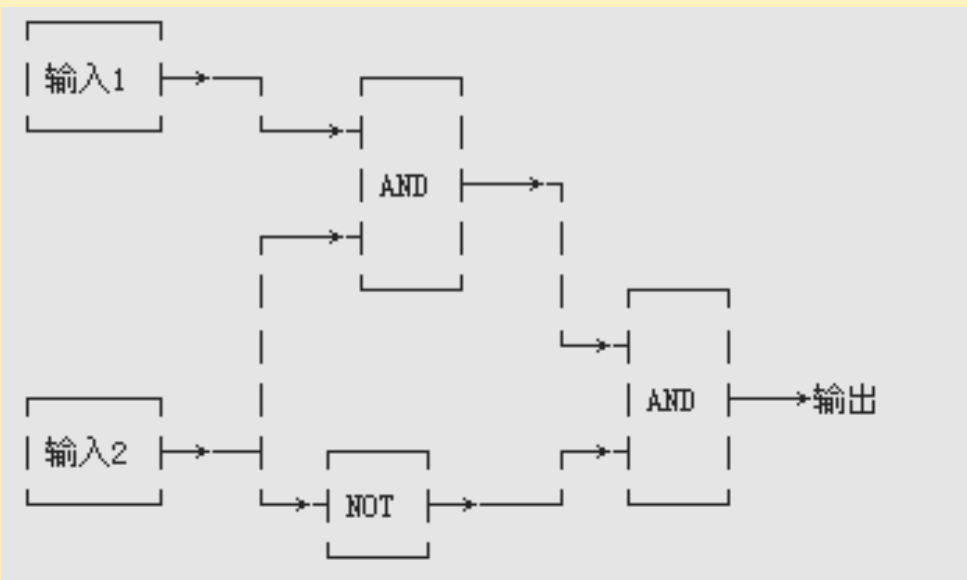
- 对于一个确定的逻辑电路，是否存在一种输入使得输出为 true，是第一个被证明的 NPC 问题。其它的 NPC 问题都是由这个问题约化而来的。
- 我们知道，一个逻辑电路由若干个输入，一个输出，若干“逻辑门”和密密麻麻的线组成，如下图：



• 这是个较简单的逻辑电路，当输入 1、输入 2、输入 3 分别为 True、True、False 或 False、True、False 时，输出为 True。

直观描述

- 有输出无论如何都不可能为 True 的逻辑电路吗？



• 这个逻辑电路中，无论输入是什么，输出都是 False。因此，这个逻辑电路不存在使输出为 True 的一组输入。



7.3. 多项式归约及 NP- 完全类

- 7.3.1 多项式归约定义
- 7.3.2 多项式归约性质
- 7.3.3 NP- 难类、 NP- 完全类
- 7.3.4 NP- 完全性证明
- 7.3.5 其他问题复杂性类



7.3.1 多项式归约 (Reduction) 定义

- 人们普遍认为, $P=NP$ 不成立 $\Rightarrow$ 多数人相信, 存在至少一个不可能有多项式级复杂度的算法的 NP 问题。
- **Reduction(“归约”或“约化”)**: 一个问题 A 可以归约为问题 B 的含义即是, 可以用解决问题 B 的解法来解决问题 A, 或者说, 问题 A 可以“变成”问题 B。
- 如: 一元一次方程可以“归约”为一元二次方程。
- 求解一个一元一次方程和求解一个一元二次方程。那么我们说, 前者可以归约为后者, 意即知道如何解一个一元二次方程那么一定能解出一元一次方程。
- 我们可以写出两个程序分别对应两个问题, 那么我们能找到一个“规则”, 按照这个规则把解一元一次方程程序的输入数据变一下, 用在解一元二次方程的程序上, 两个程序总能得到一样的结果。这个规则即是: 两个方程的对应项系数不变, 一元二次方程的二次项系数为 0。按照这个规则把前一个问题转换成后一个问题, 两个问题就等价了。

7.3.2 多项式归约性质 (Reduction)

- 问题 A 可“归约”为问题 B **直观意义**: B 的时间复杂度高于或者等于 A 的时间复杂度。也就是说, 问题 A 不比问题 B 难。
- 很显然, 归约具有一项重要的性质: **归约具有传递性**。如果问题 A 可约化为问题 B, 问题 B 可约化为问题 C, 则问题 A 一定可约化为问题 C。

定义

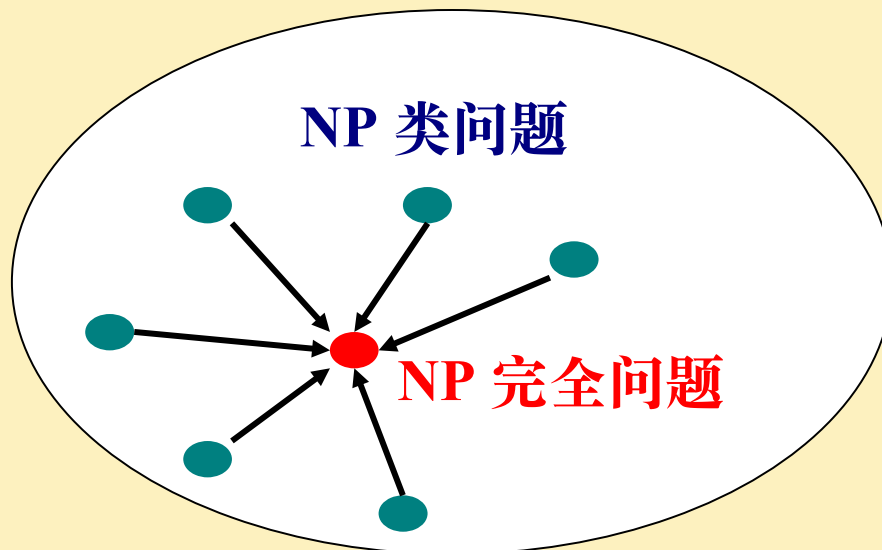
定义 10.4 设 Π 和 Π' 是两个判定问题。如果存在一个确定性算法 A , 它的行为如下, 当给 A 展示问题 Π 的一个实例 I , 算法 A 可以把它变换为问题 Π' 的实例 I' , 使得当且仅当对 I' 回答 yes 时, 对 I 回答 yes, 而且, 这个变换必须在多项式时间内完成。那么我们说 Π 多项式时间归约到 Π' , 用符号 $\Pi \propto_{poly} \Pi'$ 表示。

7.3.3 NP 难类、NP 完全类

- 术语“NP 完全”表示 NP 判定问题的一个子类，它们在下述意义上是最难的，即如果它们中的一个被证明用多项式时间确定性算法可解，那么 NP 中的所有问题用多项式时间确定性算法可解，即 $NP=P$ 。
- NP 完全问题是 **NP 类问题的子类**，一个具有特殊性质与特殊意义的子类。

NP 完全类

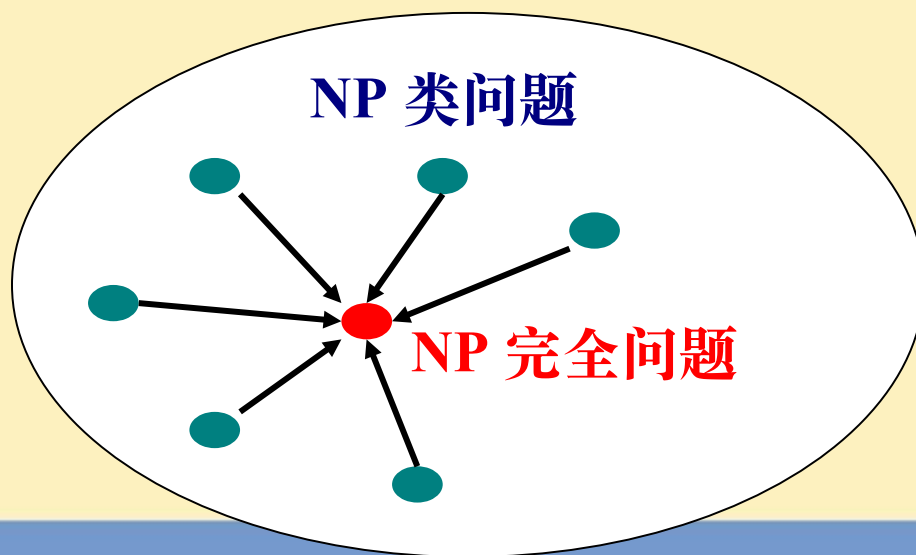
NP 完全问题是一类具备如下特殊性质的 NP 类问题



- Π (该问题本身) 就是一个 NP 类问题
- 每一个 NP 类问题都可以通过多项式时间归约到 Π

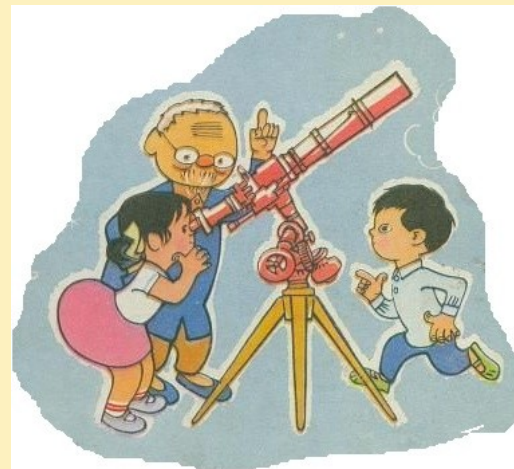
NP 完全问题的定义

- 定义 9.7(**NP 完全问题**): 令 Π 是一个判定问题, 如果问题 Π 属于 NP 类问题, 并且对 NP 类问题中的每一个问题 Π' , 都有 $\Pi' \propto_{\text{poly}} \Pi$, 则称判定问题 Π 是一个 NP 完全问题 (NP complete problem, NPC)。

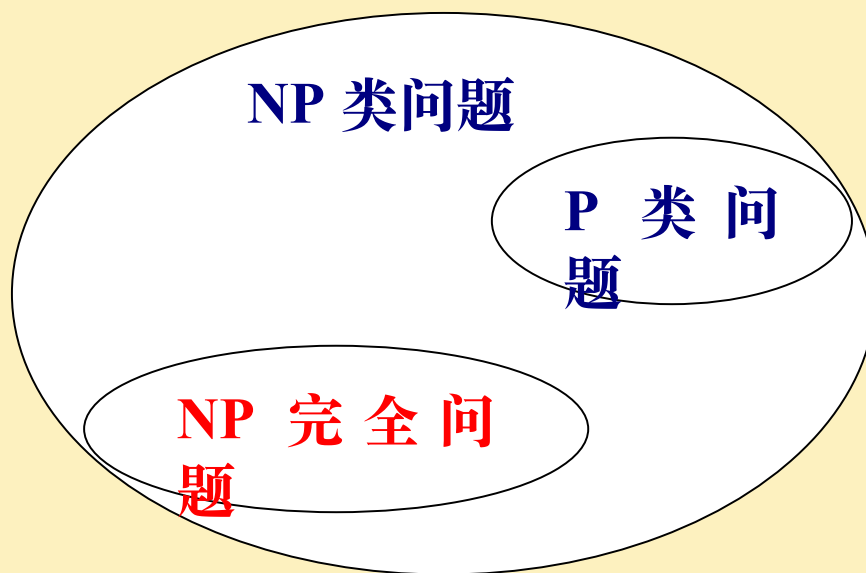


对“NP 完全问题”的评述

- NP 完全问题是 NP 类问题中最难的一类问题，至今已经发现了几千个，但一个也没有找到多项式时间算法。
- 如果某一个 NP 完全问题能在多项式时间内解决，则每一个 NP 完全问题都能在多项式时间内解决。
- 这些问题也许存在多项式时间算法，因为计算机科学是相对新生的科学，肯定还有新的算法设计技术有待发现；
- 这些问题也许不存在多项式时间算法，但目前缺乏足够的依据来证明这一点。

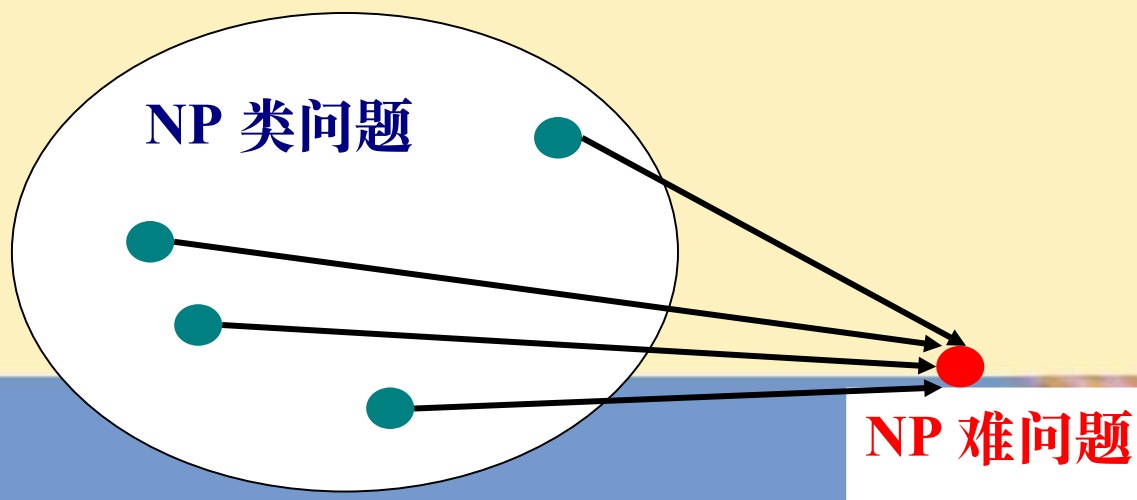


P 类问题、NP 类问题、NP 完全问题的关系



NP 困难问题

难解问题中还有一类问题，虽然也能证明所有的 NP 类问题可以在多项式时间内变换到问题 Π ，**但不能证明 Π 也是 NP 类问题**，所以 Π 不是 NP 完全的。但问题 Π 至少与任意 NP 类问题有同样的难度，这样的问题称为 NP 困难问题。



NP- 困难的 与 NP- 完全的



定义 10.5

- 一个判定问题 $\square$ 称为是 **NP 困难** 的, 如果对于 NP 中的每一个问题 $\square'$, 存在 $\square' \leq_{\text{poly}} \square$ 。



定义 10.6

- 一个判定问题 $\square$ 称为是 **NP 完全的**, 如果下列两个条件同时成立。
 - (1) $\square$ 在 NP 中,
 - (2) 对于 NP 中的每一个问题 $\square'$, $\square' \leq_{\text{poly}} \square$ 。

NP 完全问题 $\square$ 和 NP 困难问题 $\square'$ 间的差别是: $\square$ 必须是 NP 类的而 $\square'$ 可能不在 NP 中。



NP 完全问题 (补充)

- 第一个 NP 完全问题 (Cook 定理 1971)
 - 可满足性问题是 NP 完全问题
- 1972 年, Karp 证明了十几个问题都是 NP 完全的。
 - 3SAT, 3DM, VC, 团, HC, 划分
- 更多的 NP 完全问题
 - 1979 年: 300 多个
 - 1998 年: 2000 多个
- 这些 NP 完全问题的证明思想和技巧, 以及利用他们证明的几千个 NP 完全问题, 极大地丰富了 NP 完全理论。

7.3.4 NP 完全性证明



定理 (归约关系的传递性)

- 设 Π , Π' 和 Π'' 是三个判定问题, 有 $\square \square_{\text{poly}} \Pi'$ 和 $\square \square_{\text{poly}} \Pi''$, 那么 $\square \square_{\text{poly}} \Pi''$



推论 (NP 完全性的传递性)

- 如果 Π 和 Π' 是 NP 中的两个问题, 若有 $\square' \square_{\text{poly}} \Pi$, 并且 Π' 是 NP 完全的, 则 Π 是 NP 完全的。



7.3.4 NP 完全性证明

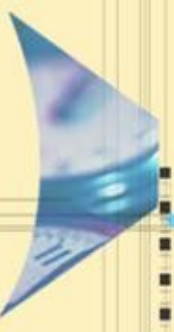
- 根据上面的推论，为了证明一个问题 Π 是 NP 完全的，仅需要证明以下两个条件同时成立

(1) $\Pi \in \text{NP}$;

(2) 存在着一个 NP 完全问题 Π' ，使 $\Pi' \leq_{\text{poly}} \Pi$ 。

NP 完全性的传递性举例

- 已知哈密顿回路问题是一个 NP 完全问题，证明旅行商问题也是一个 NP 完全问题
- 哈密顿回路问题□□：给定无向图 $G=(V,E)$ ，是否存在一条回路，使得图中每个顶点在回路中出现且只出现一次
- 旅行商问题□：给定 n 个城市和它们的距离矩阵，以及距离 L ，是否存在从某个城市出发，经过每个城市一次且仅一次，最后回到出发城市且距离小于或等于 L 的路线
- 后面是具体说明：



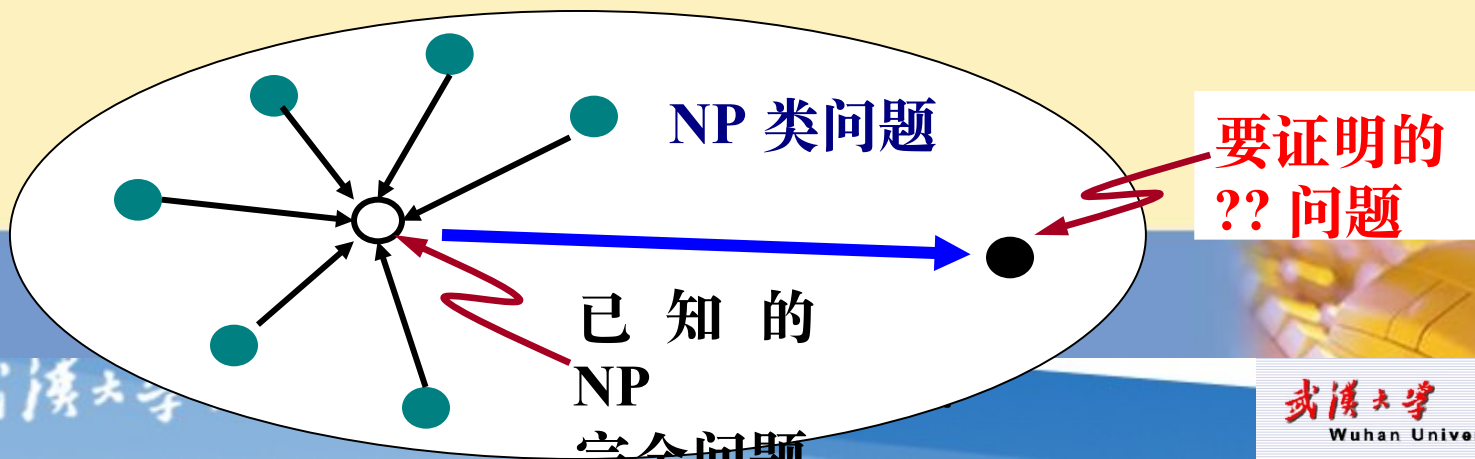
对于哈密顿回路问题□□中的无向图 $G=(V,E)$ ，可以用多项式时间构造新的无向图 $G'=(V',E')$ ，使得 $V'=V$ ， $E'=E$ 。对于 E' 中的每条边 (u,v) 赋予如下权值：

$$d(u,v) = \begin{cases} 1 & (u,v) \in E \\ 2n & (u,v) \notin E \end{cases}$$

- 通过上述转换，□□转换为旅行商问题□
- 可以证明两个问题等价：
- (1) G 中包含一条哈密顿回路，则这条路径上的边共有 n 条，每条边长度是 1，则 G' 中存在一条路径经过每个顶点且一次，并且长度不超过 n
- (2) 如果 G' 中存在一条满足旅行商问题的路径，则这条路径经过 G 中各个顶点一次，且仅一次，最后回到出发顶点，它肯定是一条哈密顿回路

如何证明一个问题是 NP 完全的？

- 根据 NP 完全问题的定义（满足的两个性质），显然地，证明需要分两个步骤进行：
 - 证明问题 II 是 NP 类问题；即可以在多项式时间内以确定性算法验证一个任意生成的串，以确定它是否为问题的一个解。
 - 证明 NP 类问题中的每一个问题都能有多项式时间内变换为问题 II。由于多项式问题变换具有传递性，所以，只需证明一个已知的 NP 完全问题能够在多项式时间内变换为问题 II。

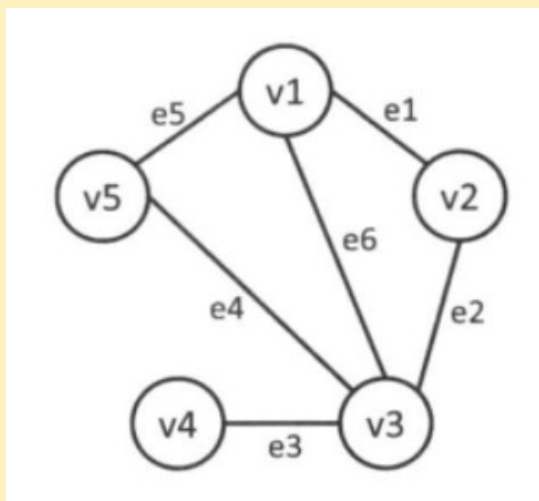


顶点覆盖、独立集和团集问题

- 顶点覆盖：
 - 给出一个无向图 $G=(V,E)$ 和一个正整数 k ，是否存在大小为 k 的子集 $C \subseteq V$ ，使 E 中的每条边至少和 C 中的一个顶点关联？
- 独立集：
 - 独立集：给出一个无向图 $G=(V,E)$ 和一个正整数 k ，是否存在 k 个顶点的子集 $S \subseteq V$ ，使得对于每一对顶点 $u, w \in S, (u,w) \notin E$ ？
- 团集：
 - 团集：给出一个无向图 $G=(V,E)$ 和一个正整数 k ， G 包含一个大小为 k 的团集吗？(注意一个 G 中大小为 k 的团集是 G 中 k 个顶点的一个完全子图。)
- 容易证明所有这三个问题确实是 NP 的。

顶点覆盖、独立集和团集问题

- 顶点覆盖：G 中的任意边的至少一个端点属于 C 的顶点集合 $C \subseteq V$
- 独立集：在 G 中两两之间互不相连的点集合
- 团集：顶点两两之间都有边的集合
- $| \text{最大独立集} | + | \text{最小点覆盖} | = |V|$



- 最小顶点覆盖为 $\{v1, v3\}$
- 最大独立集 $\{v2, v4, v5\}$
- 最大团集 $\{v1, v3, v5\}$ 、 $\{v1, v3, v2\}$

证明最大团集问题是 NP 完全的

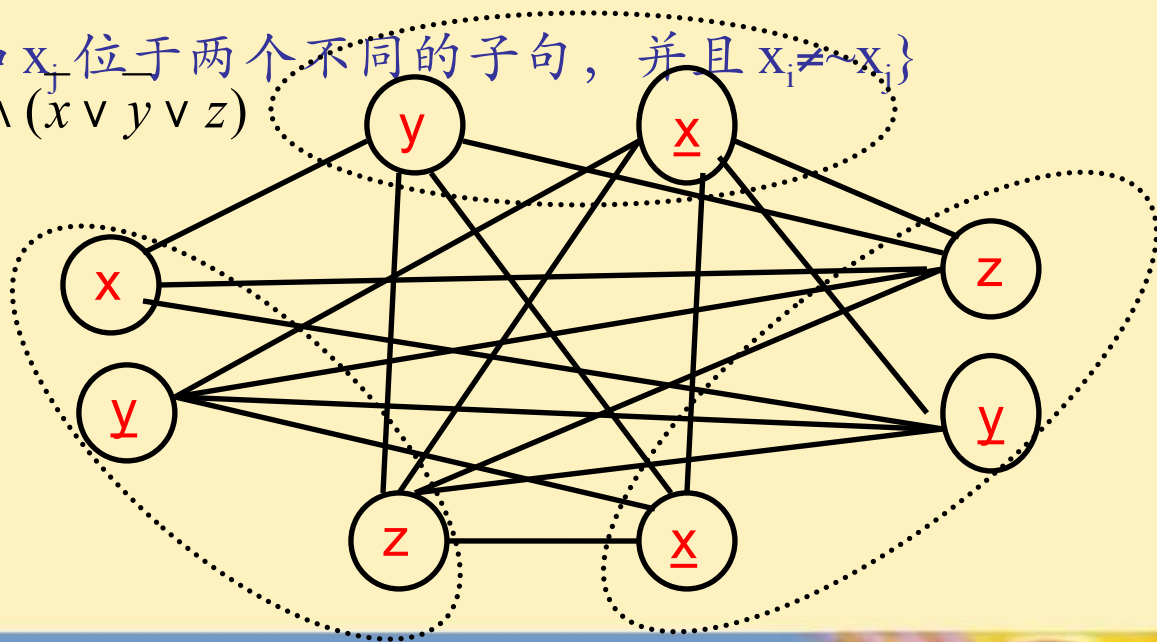
- 下面证明团问题属于 NP 完全问题，证明分两步：
 - 1) 团集问题属于 NP 类问题
 - ✓ 显然，验证图 G 的一个子图是否构成团只需要多项式时间，所以团问题属于 NP 类问题。
 - 2) 可满足性问题 $\propto_{\text{poly}}$ 团集问题



可满足性 $\square_{\text{poly}}$ 团集问题

- 给出一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，我们构造一个图 $G = (V, E)$ ，其中 V 是 $2n$ 个文字的所有出现的集合（注意一个文字是一个布尔变元或它的否定），并且

$$E = \{(x_i, x_j) \mid x_i \text{ 和 } x_j \text{ 位于两个不同的子句, 并且 } x_i \neq \neg x_j\}$$
$$f = (x \vee y \vee z) \wedge (x \vee y) \wedge (x \vee y \vee z)$$





引理 10.1

- f 是可满足的当且仅当 G 有一个大小为 m 的团集。

证明：

- 一个大小为 m 的团集对应于一个 m 个不同子句中对 m 个文字的真指派。在两个文字 a 和 b 间的边意味着当 a 和 b 同时指派真值时没有矛盾。这就得到 f 是可满足的当且仅当存在着一个对于 m 个不同子句中的 m 个文字的真指派，当且仅当 G 有一个大小为 m 的团集。

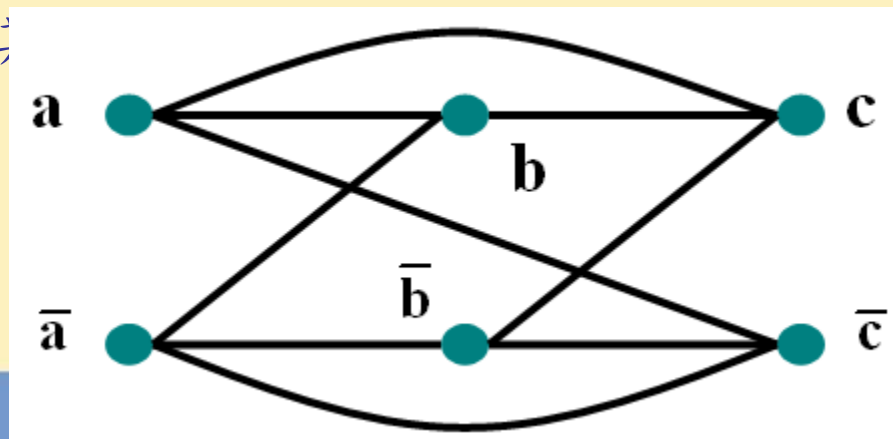


可满足性 $\propto_{\text{poly}}$ 团集问题

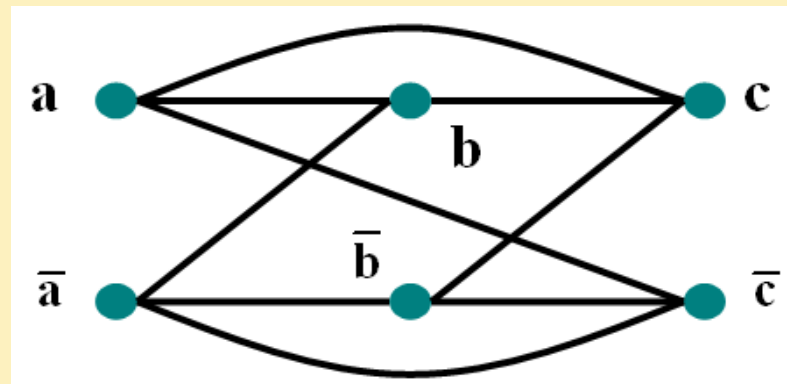
对于任意一个合取范式，按照如下方式构造相应的图 G ：

例如 $f = (a \vee \bar{b}) \wedge (b \vee \bar{c}) \wedge (c \vee \bar{a})$

- 图 G 的每个顶点对应于 f 中的每个文字，多次出现的重复表示；
- 若图 G 中两个顶点对应的文字不互补，且不出现在同一子句中，则将两顶点连边（此时为真）



SAT 问题 $\propto$ poly 团问题



设 f 有 n 个子句，则： f 可满足

- $\Leftrightarrow n$ 个子句同时为真
- $\square$ 每个子句至少 1 个文字为真 $\leftarrow$ 同时为真，则相连
- $\Leftrightarrow G$ 中有 n 个顶点之间彼此相连
- $\Leftrightarrow G$ 中有 n 个顶点的团

显然，上述构造图 G 的方法可在多项式时间内完成，故有： SAT_{poly} 团问题。

由以上证明可知，团问题是 NP 完全问题。

7.3.5 其他问题复杂性类—— co-NP 类

co-NP 类由它们的补属于 NP 类的那些问题组成。

例如：

- 旅行商问题的补：给出 n 个城市和它们之间的距离，不存在长度为 k 或更少的任何旅程，情况是那样吗？
- 可满足性问题 (SAT) 的补：给出一个公式 f ，不存在使得 f 为真的布尔变量指派，是吗？换言之， f 是不可满足的吗？

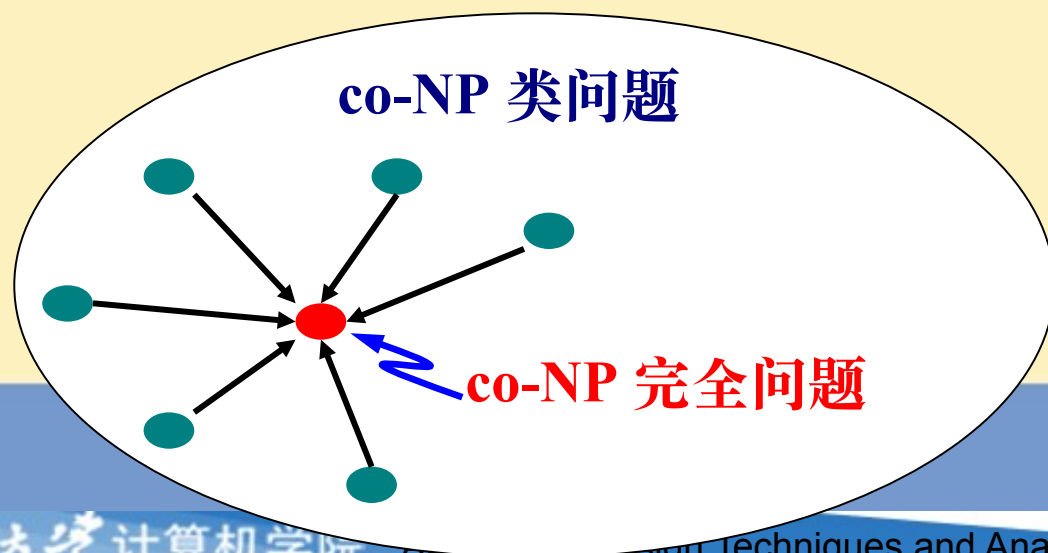
7.3.5 其他问题复杂性类——co-NP 类

- co-NP 类由它们的补属于 NP 类的那些问题组成。



定义 10.7

- 问题 Π 对于 co-NP 类是完全的，如果
 - Π 在 co-NP 中
 - 对于 co-NP 中的每一个问题 Π' ， $\Pi' \leq_{\text{poly}} \Pi$



7.3.5 其他问题复杂性类—— co-NP 类



定理 10.5

- 问题 Π 是 NP 完全的，当且仅当它的补 $\overline{\Pi}$ 对于类 co-NP 是完全的。



定理 10.6

- 重言式问题：给出一个 DNF 公式 f ，它是重言式吗？这个问题对于 co-NP 类是完全的
- 由此得出下列结论：
 - (1) 重言式问题属于 P 当且仅当 $\text{co-NP} = \text{P}$,
 - (2) 重言式问题属于 NP 当且仅当 $\text{co-NP} = \text{NP}$ 。



7.3.5 其他问题复杂性类——co-NP 类

- 如果一个复合命题，不管其原子命题取什么值，它总是为真，则我们称之为重言式。
- 要判断一个命题是不是重言式。列真值表，穷尽原子命题的所有可能取值，看看这个命题是不是总取真值，如果总是真则它就是重言式了。
- 比如判断 $(p \rightarrow q) \wedge p \rightarrow q$ 是不是重言式。

p	q	$p \rightarrow q$	$(p \rightarrow q) \wedge p$	$(p \rightarrow q) \wedge p \rightarrow q$
1	1	1	1	1
1	0	0	0	1
0	1	1	0	1
0	0	1	0	1

7.3.5 其他问题复杂性类——NPI 类

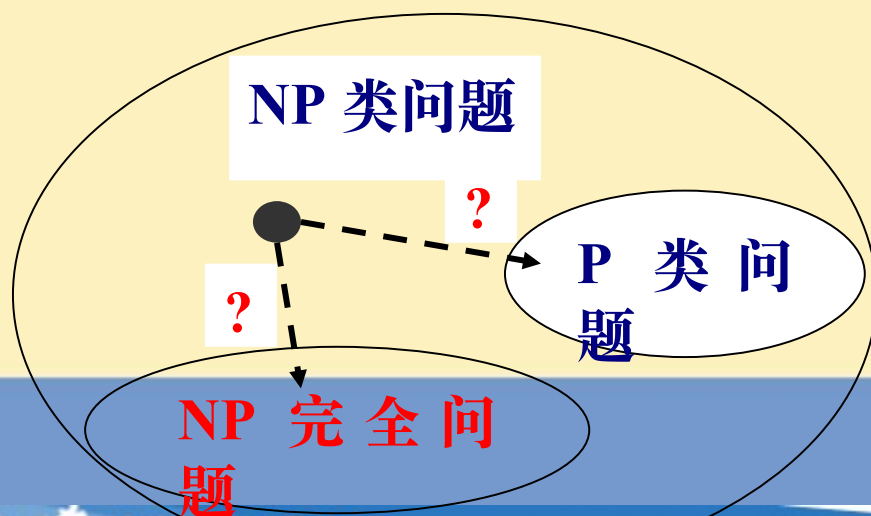


定理 10.7

- 如果问题 Π 和它的补 $\overline{\Pi}$ 是 NP 完全的，那么 $\text{co-NP} = \text{NP}$ 。
- 换句话说，如果问题 Π 和它的补都是 NP 完全的，那么 NP 类在补运算下是封闭的。
- NPI class: the class of problems that they and their complementation are in NP, but are not NP-complete.

7.3.5 其他问题复杂性类—— NPI 类

- NP 类问题中还有一些问题，人们不知道是属于 P 类还是属于 NP 完全问题，还有待于证明其归属。
- 这些问题是 NP 完全问题的可能性非常小，也因为不相信他们在 P 中，我们人为地增加另一问题类来接纳这类问题，这个类称为 NPI(NP-Intermediate) 类。



7.3.5 其他问题复杂性类—— NPI 类

- NPI 类是一个人为定义的、动态的概念，随着人们对问题研究的深入，许多 NPI 类问题逐渐被明白无误地证明他们原本属于 P 类问题或 NP 完全问题。
- 例如：线性规划问题、素数判定问题等，在二者没有被证明他们均属于 P 类问题之前，人们一直将他们归于 NPI 类问题。

一个例子

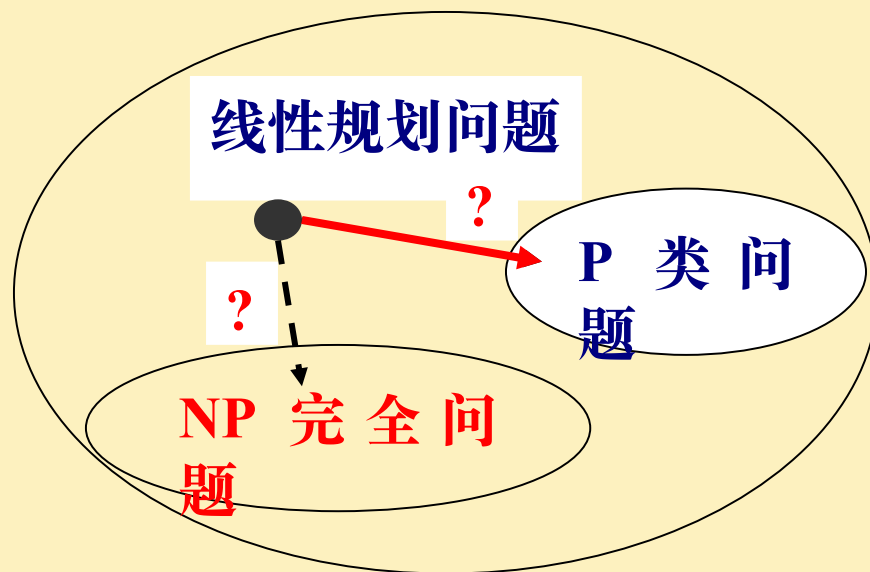
- 线性规划问题

设 $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$, $b \in \mathbb{R}^m$,

在满足 $Ax=b$, $x \geq 0$ 约束下,

使目标函数 $c^T x$ 达到最大值, 其中 $c \in \mathbb{R}^n$.

- 长期以来, 线性规划问题没有多项式时间解法, 也无法证明它是 NP 完全问题。



线性规划问题

- 直到 20 世纪 80 年代，这个问题得到解决，发现了多项式时间算法。
- 1984 年，贝尔实验室印度裔数学家卡马卡（Narendra Karmarkar）提出了投影尺度法（又名 Karmarkar's algorithm）。这是第一个在理论上和实际上都表现良好的算法：它的最坏情况仅为多项式时间，且在实际问题中它比单纯形算法有显著的效率提升。
- 线性规划的求解程式在各种各样的工业最优化问题里被广泛使用，例如运输网络的流量的最优化问题，其中很多都可以不太困难地被转换成线性规划问题。

线性规划问题

以下是一个线性规划的例子。假设一个农夫有一块 A 平方千米的农地，打算种植小麦或大麦，或是两者依某一比例混合种植。该农夫只可以使用有限数量的肥料 F 和农药 P ，而单位面积的小麦和大麦都需要不同数量的肥料和农药，小麦以 (F_1, P_1) 表示，大麦以 (F_2, P_2) 表示。设小麦和大麦的售出价格分别为 S_1 和 S_2 ，则小麦与大麦的种植面积问题可以表示为以下的线性规划问题：

$$\max Z = S_1 x_1 + S_2 x_2 \text{ (最大化利润 - 目标函数)}$$

$$s. t. x_1 + x_2 \leq A \text{ (种植面积的限制)}$$

$$F_1 x_1 + F_2 x_2 \leq F \text{ (肥料数量的限制)}$$

$$P_1 x_1 + P_2 x_2 \leq P \text{ (农药数量的限制)}$$

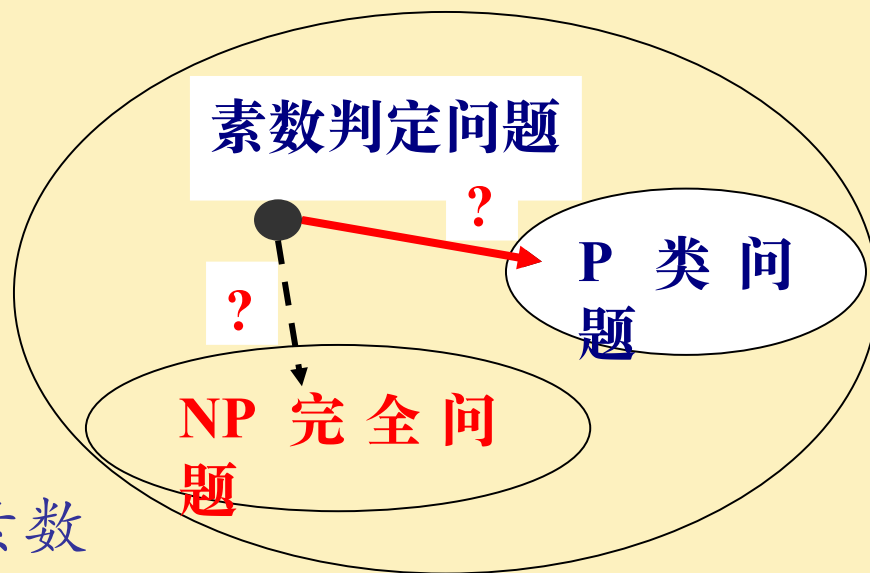
$$x_1 \geq 0, x_2 \geq 0 \text{ (不可以栽种负数的面积)}$$

另一个例子

素数判定问题

- 给一个整数，判定其是素数还是合数。
- 经过一个重要的理论突破，印度 M. Agrawal 教授和他的学生 N. Kayal 和 N. Saxena 于 2002 年宣布发现一个多项式时间算法。

(PRIMES is in P, *Annals of Mathematics*, 160 (2004), 781–793)



四种类之间的关系

- P184

